



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

DataQuest: Plataforma Web, API y Extensión de Escritorio IDE para la Validación de Normalización de Bases de Datos Relacionales

Informe de Especificación de Requerimientos de Software (SRS)

FD03 - Versión final alineada al formato académico de referencia

Curso: Base de Datos II

Docente: Ing. Patrick Jose Cuadros Quiroga

Integrantes:

Dongo Panza, Manuel Andre (2023076802)

**Tacna - Perú
2026**



CONTROL DE VERSIONES

Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	MADP	Docente / revisión de curso	Pendiente	21/05/2026	Versión base de documentación del proyecto Validador de Normalización.
2.0	MADP / Auditoría técnica asistida	Pendiente de revisión docente	Pendiente	10/06/2026	Reconstrucción alineada a DataQuest Web, API, Supabase y Extensión VS Code.
3.0	MADP / Revisión FD03 final	Pendiente de validación	Pendiente	25/06/2026	Versión final con formato FD03, requerimientos, diagramas, trazabilidad, API y criterios de aceptación.

DataQuest Web + API de Normalización + Extensión VS Code

Documento de Especificación de Requerimientos de Software

Versión 3.0 final



ÍNDICE GENERAL

INTRODUCCIÓN	4
I. Generalidades del sistema	5
A. Nombre del sistema	5
B. Visión	5
C. Misión.....	5
D. Organigrama funcional	5
II. Visionamiento del sistema	6
A. Descripción del sistema	6
B. Objetivos de negocio	6
C. Objetivos de diseño.....	6
D. Alcance del proyecto	6
E. Viabilidad del sistema	7
F. Información obtenida del levantamiento de información	7
G. Matriz de consistencia sistema-documento.....	7
III. Especificación de Requerimientos de Software	9
A. Cuadro de Requerimientos Funcionales Inicial	9
B. Cuadro de Requerimientos No Funcionales	10
C. Cuadro de Requerimientos Funcionales Final.....	11
D. Reglas de Negocio	12
IV. Fase de Desarrollo.....	13
A. Perfiles de Usuario	13
Usuario estudiante / diseñador de base de datos	13
Usuario desarrollador	13
Administrador académico.....	13
Docente / Evaluador	13
API FastAPI	13
Base de datos Supabase/PostgreSQL.....	13
B. Diagrama de Componentes	13
C. Diagrama de Despliegue	14
D. Diagrama de Estados - Validación de esquema	14



E. Casos de Uso - General	14
F. Casos de Uso - Módulo Web.....	16
G. Casos de Uso - Módulo API	16
H. Modelo Lógico	17
J. Análisis de Objetos	17
H. Diagramas de Actividades, Secuencia y Clases	17
V. API de integración y endpoints.....	18
VI. Matriz de trazabilidad	18
VII. Criterios de calidad, seguridad y aceptación	19
A. Criterios de calidad	19
B. Criterios de seguridad	19
CONCLUSIONES	20
RECOMENDACIONES	20
BIBLIOGRAFÍA.....	20

INTRODUCCIÓN

El presente Informe de Especificación de Requerimientos de Software (SRS) describe de manera ordenada los requerimientos funcionales, requerimientos no funcionales, reglas de negocio, perfiles de usuario, casos de uso, modelo lógico, diagramas, API, criterios de calidad y criterios de aceptación del sistema DataQuest.

DataQuest se documenta como una solución académica orientada al análisis de esquemas de bases de datos relacionales, con capacidad para detectar incumplimientos de Primera Forma Normal, Segunda Forma Normal y Tercera Forma Normal mediante reglas heurísticas implementadas en Python.

El sistema integra una interfaz web desarrollada con Streamlit, una API FastAPI para consumo externo, una extensión de Visual Studio Code para análisis desde el editor, un motor de análisis compuesto por parser, diagnóstico, corrector y generadores de reporte, además de servicios de persistencia asociados a Supabase/PostgreSQL.

El objetivo de este documento es mantener trazabilidad entre el repositorio, la arquitectura, los módulos existentes, los requerimientos, los casos de uso, la API y los criterios de validación docente, evitando contradicciones entre lo implementado, lo planificado y lo documentado.

I. Generalidades del sistema

A. Nombre del sistema

DataQuest: Plataforma Web, API y Extensión de Escritorio IDE para la Validación de Normalización de Bases de Datos Relacionales.

B. Visión

Ser una plataforma académica y técnica que facilite el aprendizaje, análisis y mejora de esquemas de bases de datos relacionales, integrando una experiencia web, servicios API y herramientas de asistencia para desarrolladores en entornos de programación.

C. Misión

Brindar una solución clara, verificable y educativa que permita cargar esquemas, diagnosticar el nivel de normalización, explicar violaciones, sugerir correcciones, generar reportes y conservar trazabilidad histórica de validaciones.

D. Organigrama funcional

El ecosistema funcional de DataQuest se organiza alrededor del usuario web, el evaluador académico, el consumidor de API, el motor de análisis y la base de datos de soporte. La siguiente figura resume la relación entre actores y componentes principales.

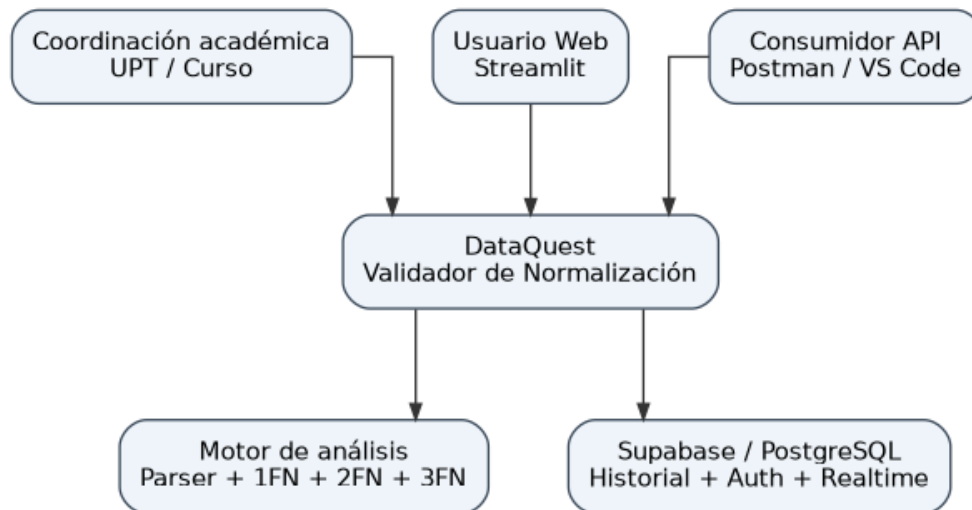


Figura 1. Organigrama funcional del ecosistema DataQuest.



II. Visionamiento del sistema

A. Descripción del sistema

DataQuest es una plataforma de apoyo educativo y técnico para validar la normalización de esquemas relacionales. La interfaz web permite al usuario ingresar SQL o cargar archivos con estructuras de tablas. El motor de análisis interpreta tablas, claves, atributos y posibles dependencias para determinar el cumplimiento de 1FN, 2FN y 3FN.

La API permite que clientes externos, Postman, integraciones web o la extensión de Visual Studio Code envíen esquemas SQL y reciban un diagnóstico JSON estructurado. La persistencia utiliza Supabase/PostgreSQL para autenticación, historial, perfiles y datos de comunidad cuando el módulo se encuentre habilitado.

B. Objetivos de negocio

- Facilitar el aprendizaje de normalización de bases de datos mediante diagnósticos claros y explicaciones comprensibles.
- Reducir errores de diseño relacional antes de la implementación física de una base de datos.
- Centralizar validaciones, resultados, historial y reportes en una plataforma académica verificable.
- Permitir integración mediante API para otros sistemas, pruebas con Postman y extensión de Visual Studio Code.
- Documentar el proyecto con trazabilidad entre requerimientos, módulos, entidades, endpoints y criterios de aceptación.
- Evitar que el sistema se interprete como reemplazo de una revisión profesional exhaustiva; su enfoque es educativo y asistido.

C. Objetivos de diseño

- Aplicar arquitectura por capas: vistas, controladores, motor de dominio, API, persistencia y servicios de visualización.
- Mantener separación entre la interfaz Streamlit, la API FastAPI, la lógica de normalización y la capa Supabase/PostgreSQL.
- Exponer respuestas JSON claras para éxito y error, con estructura consistente para integraciones externas.
- Soportar entrada por texto SQL y archivos compatibles como SQL, TXT, CSV y XLS/XLSX según el módulo.
- Generar reportes descargables y sugerencias de corrección sin sobrescribir información del usuario sin autorización.
- Documentar diagramas, requerimientos, reglas de negocio y matriz de trazabilidad bajo el formato FD03/SRS.

D. Alcance del proyecto

El alcance incluye la aplicación web, el motor de análisis, la API de normalización, la persistencia en Supabase/PostgreSQL, la extensión VS Code, reportes, visualización de diagramas y documentación



académica. No incluye modelamiento físico automático definitivo, decisiones vinculantes de diseño de datos ni reemplazo de revisión experta.

- Incluye registro e inicio de sesión cuando el módulo de autenticación Supabase esté habilitado.
- Incluye validación de esquemas relacionales por 1FN, 2FN y 3FN mediante reglas programadas.
- Incluye diagnóstico de violaciones, sugerencias de mejora y explicación pedagógica del resultado.
- Incluye endpoint POST /api/normalizacion para integraciones externas.
- Incluye generación de reportes SQL/PDF cuando el flujo lo permita.
- Incluye historial de validaciones y dashboard de resultados.
- Incluye extensión de VS Code para inyectar reportes en archivos del editor.

E. Viabilidad del sistema

Tipo de viabilidad	Evaluación
Técnica	Alta para demo académica. El sistema se apoya en Python 3.10+, Streamlit, FastAPI, Supabase/PostgreSQL, Graphviz, Docker, TypeScript y VS Code API.
Operativa	Media-alta. La web guía al usuario; la API permite pruebas controladas; la extensión amplía el alcance hacia desarrolladores. Requiere explicar claramente que el diagnóstico es asistido.
Económica	Viable para entorno académico. Las tecnologías principales tienen opciones gratuitas o de bajo costo. Los costos se concentran en despliegue, base de datos y mantenimiento.
Académica	Alta. El proyecto permite demostrar análisis de bases de datos, arquitectura, integración API, pruebas, seguridad y documentación SRS.
Integración	Media-alta. Existen endpoints y clientes documentados; se requiere validar consistencia de payloads, manejo de errores y autenticación en entornos reales.

F. Información obtenida del levantamiento de información

- Los estudiantes requieren ejemplos concretos para diferenciar 1FN, 2FN y 3FN.
- El sistema debe permitir ingresar SQL directamente o cargar archivos compatibles sin romper el flujo.
- La respuesta del motor debe ser explicativa y no limitarse a indicar un nivel de normalización.
- Las violaciones deben asociarse a tabla, atributo, forma normal afectada y sugerencia de corrección.
- La API debe poder verificarse con Postman y ser consumible por extensiones o frontends externos.
- La persistencia debe proteger el historial del usuario y evitar exposición de esquemas ajenos.

G. Matriz de consistencia sistema-documento

Elemento encontrado en sistema	Archivo o módulo donde existe	Debe aparecer en FD03	Sección donde se documenta
Interfaz web Streamlit	app.py, views/00_inicio.py, views/04_validador.py	Sí	Introducción, RF, Casos de uso
Autenticación y sesiones	controllers/auth_controller.py, db/auth.py	Sí	RF, RNF, Seguridad
Motor parser	core/parser.py	Sí	Modelo lógico, Objetos, API
Diagnóstico 1FN-3FN	core/diagnostico.py, core/validador_1fn.py, validador_2fn.py, validador_3fn.py	Sí	RF, Reglas, Actividades



Corrector y sugerencias	core/corrector.py	Sí	RF, Casos de uso, Objetos
Generador de reportes	core/generador_pdf.py, core/generador_sql.py	Sí	RF, RNF, Entregables
API FastAPI	api.py, INTEGRACION_API.md	Sí	API, Endpoints, Pruebas
Persistencia Supabase	db/conexion.py, db/modelos.sql, db/realtime.py	Sí	Modelo lógico, Despliegue
Extensión VS Code	vscode- extension/package.json, TypeScript	Sí	Alcance, Casos de uso, API
Docker y despliegue	Dockerfile, terraform/, GitHub Actions	Sí	Despliegue, RNF, Viabilidad



III. Especificación de Requerimientos de Software

A. Cuadro de Requerimientos Funcionales Inicial

ID	Descripción	Prioridad
RF-01	Autenticación de usuarios mediante Supabase Auth para acceso a módulos privados.	Alta
RF-02	Ingreso de esquema SQL por editor de texto en la interfaz web.	Alta
RF-03	Carga de archivos SQL, TXT, CSV y XLS/XLSX para análisis de estructura.	Alta
RF-04	Parseo de tablas, columnas, claves primarias y claves foráneas.	Alta
RF-05	Validación de Primera Forma Normal con detección de atributos no atómicos o repetitivos.	Alta
RF-06	Validación de Segunda Forma Normal con detección de dependencias parciales.	Alta
RF-07	Validación de Tercera Forma Normal con detección de dependencias transitivas.	Alta
RF-08	Generación de sugerencias de corrección estructural.	Alta
RF-09	Presentación del nivel actual de normalización y resumen del diagnóstico.	Alta
RF-10	Generación de reportes SQL y PDF cuando corresponda.	Media
RF-11	Registro de historial de validaciones por usuario autenticado.	Alta
RF-12	Dashboard con métricas de validaciones, errores y avance.	Media
RF-13	Módulo de comunidad con usuarios conectados mediante realtime.	Media
RF-14	API REST para validar esquemas mediante POST /api/normalizacion.	Alta
RF-15	Endpoint raíz para verificar estado del servicio.	Media
RF-16	Respuesta JSON estándar para éxito y error.	Alta
RF-17	Manejo de errores 400 y 500 sin romper el cliente.	Alta
RF-18	Extensión VS Code para enviar SQL al motor e insertar reporte en el archivo.	Alta
RF-19	CLI para ejecución local y agentes IA.	Media
RF-20	Visualización de diagramas de tablas y relaciones.	Media
RF-21	Control de acceso al historial propio del usuario.	Alta
RF-22	Validación de entrada vacía, sintaxis inválida y ausencia de tablas.	Alta
RF-23	Exportación o descarga de resultados cuando el módulo esté habilitado.	Media
RF-24	Documentación técnica de endpoints para Postman.	Alta
RF-25	Separación de capas: vistas, controladores, core, db, visualización y API.	Alta
RF-26	Compatibilidad de despliegue mediante Docker.	Media
RF-27	Configuración mediante variables de entorno.	Alta
RF-28	Persistencia de datos de usuario y resultados en PostgreSQL/Supabase.	Alta
RF-29	Generación de mensajes pedagógicos por forma normal.	Alta
RF-30	Registro de trazabilidad entre requerimientos, módulos y criterios de aceptación.	Alta



B. Cuadro de Requerimientos No Funcionales

ID	Categoría	Requerimiento No Funcional	Prioridad
RNF-01	Seguridad	El sistema debe proteger módulos privados mediante autenticación y control de sesión.	Crítica
RNF-02	Privacidad	Cada historial de validación debe pertenecer a su usuario y no exponerse a terceros.	Crítica
RNF-03	Interoperabilidad	La API debe responder JSON y no HTML en endpoints de integración.	Alta
RNF-04	Rendimiento	Las validaciones simples deben responder en menos de 3 segundos en entorno de demostración.	Media
RNF-05	Usabilidad	La interfaz debe presentar mensajes claros, formularios comprensibles y resultados ordenados.	Alta
RNF-06	Mantenibilidad	El código debe separar vistas, controladores, core, base de datos y visualización.	Alta
RNF-07	Portabilidad	El sistema debe ejecutarse con Python 3.10+, dependencias de requirements y Docker.	Media
RNF-08	Compatibilidad	La web debe funcionar en navegadores modernos y la extensión en Visual Studio Code.	Media
RNF-09	Validación	La API debe validar entrada vacía, JSON mal formado y ausencia de tablas.	Alta
RNF-10	Error handling	Los errores deben informarse de forma controlada sin exponer stacktrace al usuario final.	Alta
RNF-11	Trazabilidad	Cada requerimiento debe vincularse con módulo, caso de uso, endpoint o criterio de aceptación.	Alta
RNF-12	Auditoría	Las operaciones relevantes deben poder asociarse a usuario, fecha, esquema y resultado.	Media
RNF-13	Escalabilidad	El historial y listados deben admitir paginación o filtros en futuras versiones.	Media
RNF-14	Accesibilidad	Los textos deben ser comprensibles para estudiantes y usuarios principiantes.	Media
RNF-15	Codificación	El sistema debe usar UTF-8 para evitar pérdida de caracteres en SQL y reportes.	Alta
RNF-16	Configuración	Credenciales y URL de Supabase no deben quedar quemadas en el código fuente.	Alta
RNF-17	Disponibilidad	El sistema debe mostrar mensajes controlados cuando falle la API o la base de datos.	Alta
RNF-18	Pruebas	Los endpoints principales deben poder probarse desde Postman.	Alta
RNF-19	Educativo	El diagnóstico debe indicarse como asistencia académica y no como decisión absoluta.	Crítica
RNF-20	Documentación	El proyecto debe mantener README, guía de API y FD03 actualizados.	Alta



C. Cuadro de Requerimientos Funcionales Final

ID	Requerimiento	Módulo	Estado	Prioridad
RF-01	El sistema debe permitir acceso web a la interfaz principal de DataQuest.	Web	Implementado	Alta
RF-02	El sistema debe permitir ingresar SQL directamente en el módulo validador.	Web/Core	Implementado	Alta
RF-03	El sistema debe interpretar sentencias CREATE TABLE mediante parser.	Core	Implementado	Alta
RF-04	El sistema debe diagnosticar el cumplimiento de 1FN.	Core	Implementado	Alta
RF-05	El sistema debe diagnosticar el cumplimiento de 2FN.	Core	Implementado	Alta
RF-06	El sistema debe diagnosticar el cumplimiento de 3FN.	Core	Implementado	Alta
RF-07	El sistema debe devolver violaciones separadas por forma normal.	Core/API	Implementado	Alta
RF-08	El sistema debe generar mejoras opcionales y sugerencias de corrección.	Core	Implementado	Alta
RF-09	El sistema debe exponer GET / para verificar estado de la API.	API	Implementado	Media
RF-10	El sistema debe exponer POST /api/normalizacion para validación.	API	Implementado	Alta
RF-11	El sistema debe devolver success, nivel_actual, violaciones y resumen.	API	Implementado	Alta
RF-12	El sistema debe manejar entrada SQL vacía con respuesta controlada.	API	Implementado	Alta
RF-13	El sistema debe manejar ausencia de tablas válidas.	API	Implementado	Alta
RF-14	El sistema debe permitir ejecución por CLI para agentes o scripts.	CLI	Implementado	Media
RF-15	El sistema debe permitir extensión VS Code para análisis desde editor.	IDE/API	Implementado/Parcial	Alta
RF-16	El sistema debe generar reportes PDF o SQL cuando el flujo lo requiera.	Core/Web	Parcial	Media
RF-17	El sistema debe almacenar historial de validaciones por usuario.	DB/Web	Parcial	Alta
RF-18	El sistema debe habilitar autenticación mediante Supabase.	DB/Web	Parcial	Alta
RF-19	El sistema debe presentar dashboard de métricas.	Web	Parcial	Media
RF-20	El sistema debe mostrar comunidad o presencia en tiempo real.	Web/DB	Planificado/Parcial	Media
RF-21	El sistema debe desplegarse mediante Docker o App Service.	Infraestructura	Parcial	Media
RF-22	El sistema debe mantener documentación de integración API.	Documentación	Implementado	Alta
RF-23	El sistema debe validar y controlar errores 400/500.	API	Implementado/Parcial	Alta
RF-24	El sistema debe mantener separación MVC extendida.	Arquitectura	Obligatorio	Alta
RF-25	El sistema debe registrar trazabilidad de requerimientos.	Documentación	Implementado	Alta



D. Reglas de Negocio

- DataQuest es una herramienta académica de asistencia para normalización; no reemplaza una revisión profesional completa.
- El usuario debe ingresar un esquema relacional con tablas, columnas y claves suficientes para que el diagnóstico sea útil.
- La ausencia de claves primarias o dependencias explícitas puede obligar al motor a usar inferencias heurísticas.
- Toda sugerencia debe considerarse recomendación técnica y no modificación automática obligatoria.
- El historial de validaciones debe pertenecer al usuario autenticado que lo generó.
- La API debe rechazar entradas vacías o sin tablas válidas mediante respuesta controlada.
- Los reportes no deben incluir credenciales, variables de entorno ni datos sensibles.
- La extensión VS Code no debe alterar el archivo fuente sin acción explícita del usuario.
- La base de datos de persistencia debe utilizar políticas de acceso cuando se almacene información de usuarios.
- El sistema debe diferenciar funciones implementadas, parciales y planificadas para evitar contradicciones ante revisión docente.

IV. Fase de Desarrollo

A. Perfiles de Usuario

Usuario estudiante / diseñador de base de datos

Usa la web para cargar esquemas, revisar violaciones, aprender formas normales y descargar resultados.

Usuario desarrollador

Consume la API o la extensión VS Code para validar esquemas durante el desarrollo.

Administrador académico

Revisa métricas, historial, evidencias de funcionamiento y trazabilidad del proyecto.

Docente / Evaluador

Valida consistencia entre sistema, documentación, pruebas, arquitectura y alcance.

API FastAPI

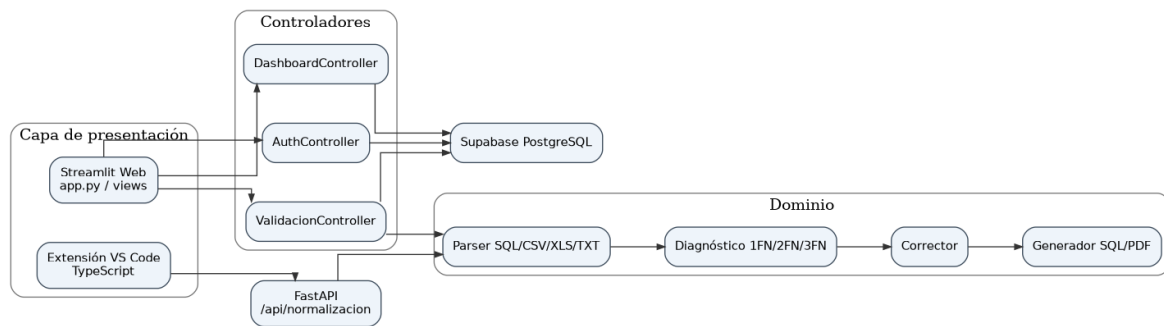
Componente técnico que recibe SQL, valida entrada, ejecuta parser y diagnóstico, y responde JSON.

Base de datos Supabase/PostgreSQL

Almacena usuarios, historiales, perfiles y datos de comunidad cuando estén habilitados.

B. Diagrama de Componentes

El diagrama de componentes muestra la separación entre presentación, controladores, motor de análisis, API y persistencia. Esta separación evita mezclar interfaz con reglas de diagnóstico y facilita pruebas por módulo.



Fuente: elaboración propia a partir del repositorio DataQuest y del formato académico FD03/SRS, 2026.

Figura 2. Diagrama de componentes de DataQuest.

C. Diagrama de Despliegue

El despliegue separa clientes, servicios web/API, base de datos y herramientas de validación. La extensión VS Code y Postman consumen la API por HTTP/JSON; la aplicación web puede interactuar con API y Supabase según el flujo configurado.

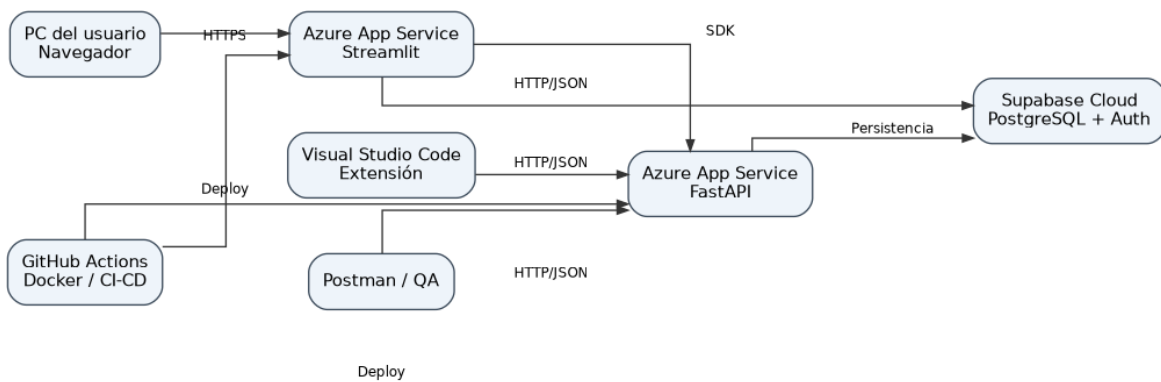


Figura 3. Diagrama de despliegue de DataQuest.

D. Diagrama de Estados - Validación de esquema

La validación de un esquema sigue estados controlados desde la recepción hasta el registro del historial. Cada transición depende de que el formato sea válido y que el motor pueda extraer tablas y relaciones mínimas.



Figura 4. Diagrama de estados del proceso de validación.

E. Casos de Uso - General

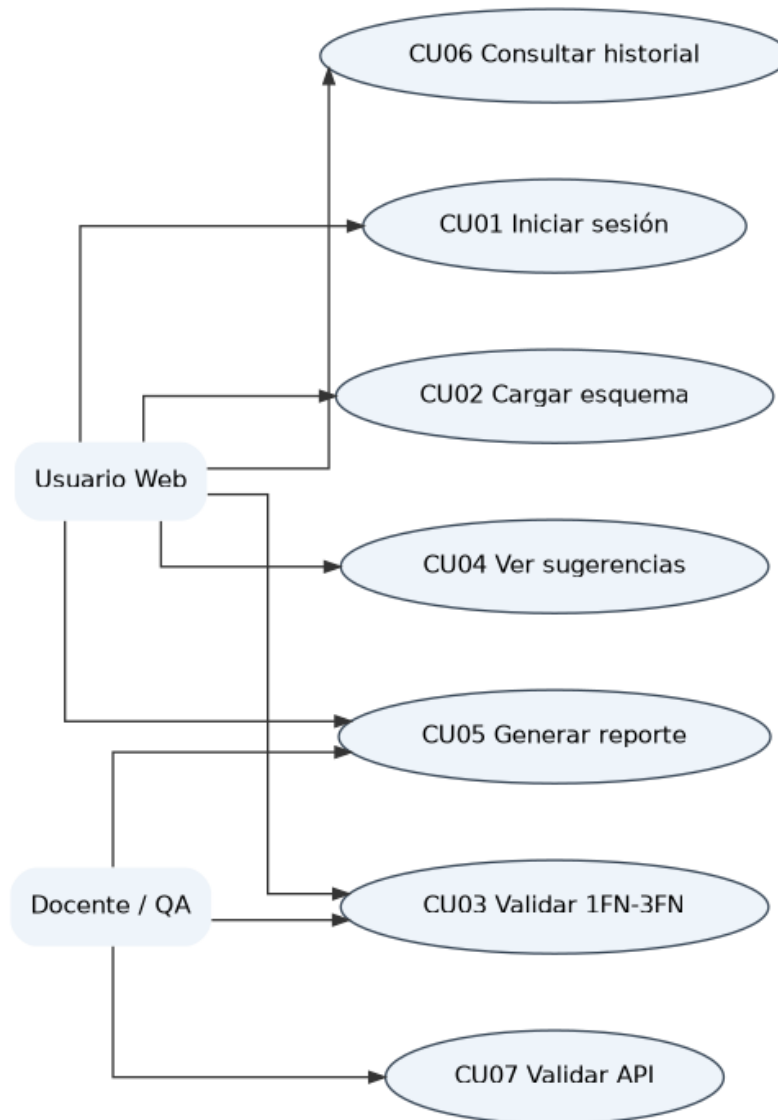


Figura 5. Diagrama general de casos de uso.

CU	Caso de uso	Actor principal	Descripción resumida
CU01	Iniciar sesión	Usuario Web	Valida credenciales para acceder a módulos privados.
CU02	Cargar esquema	Usuario Web	Ingresa SQL o archivo compatible para análisis.
CU03	Diagnosticar normalización	Usuario Web	Ejecuta validación de 1FN, 2FN y 3FN.
CU04	Consultar sugerencias	Usuario Web	Revisa mejoras propuestas por el motor.
CU05	Generar reporte	Usuario Web	Descarga o visualiza resultados técnicos.
CU06	Consultar historial	Usuario Web	Revisa validaciones anteriores.
CU07	Validar API	Postman / QA	Prueba endpoints, JSON y manejo de errores.

F. Casos de Uso - Módulo Web

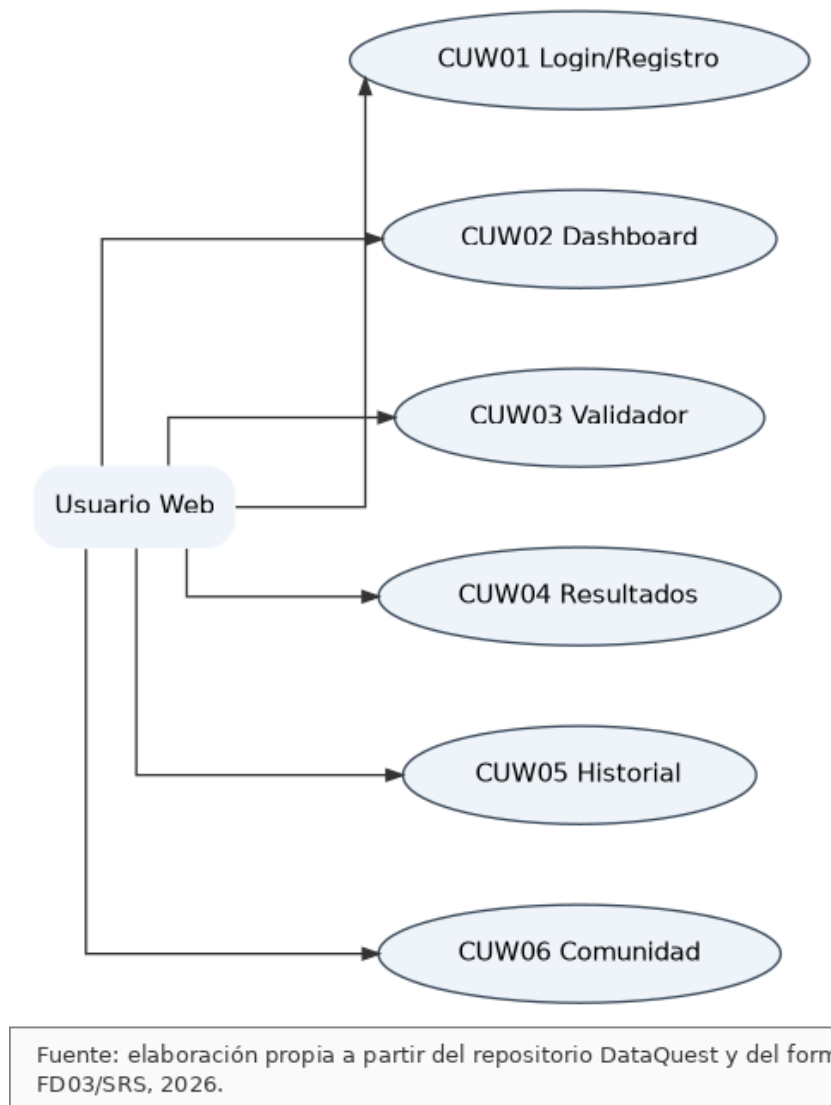


Figura 6. Casos de uso del módulo Web.

G. Casos de Uso - Módulo API

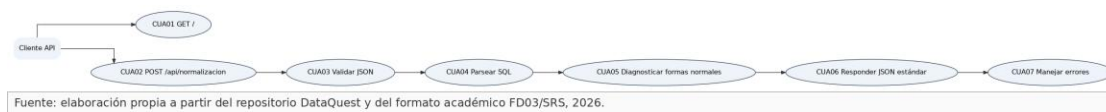


Figura 7. Casos de uso del módulo API.

H. Modelo Lógico

El modelo lógico agrupa las entidades centrales del sistema: usuario, validación, tabla, columna, violación, sugerencia, reporte, historial, sesión y configuración de API.

Entidad	Descripción	Atributos principales	Relaciones
Usuario	Cuenta del sistema.	id, email, nombre, rol, estado	Validacion, Historial
Validacion	Análisis realizado sobre un esquema.	id, usuarioId, esquemaOriginal, nivelActual, fecha	Usuario, Tabla, Violacion
Tabla	Tabla detectada en el esquema.	id, validacionId, nombre, pk, fk	Validacion, Columna
Columna	Atributo de una tabla.	id, tablaId, nombre, tipo, esPk, esFk	Tabla
Violacion	Incumplimiento de forma normal.	id, validacionId, formaNormal, tabla, mensaje	Validacion, Sugerencia
Sugerencia	Recomendación de corrección.	id, violacionId, descripcion, sqlPropuesto	Violacion
Reporte	Resultado exportable.	id, validacionId, tipo, rutaSegura	Validacion
Sesion	Control de acceso.	token, usuarioId, expiracion	Usuario
ApiLog	Registro técnico de consumo API.	id, endpoint, estado, fecha	API
Configuracion	Variables y parámetros.	clave, valor, ambiente	Sistema

J. Análisis de Objetos

Objeto	Responsabilidad	Atributos	Métodos / operaciones	Relación
Usuario	Representa una cuenta que accede al sistema.	id, email, rol	autenticar, cerrarSesion, actualizarPerfil	Validacion
Parser	Interpreta entrada SQL o archivo.	contenido, formato	parse_schema, extraer_tablas	Diagnostico
Diagnostico	Evalúa el cumplimiento de formas normales.	tablas, dependencias	validar_1fn, validar_2fn, validar_3fn	Violacion
Corrector	Propone mejoras estructurales.	violaciones, esquema	generar_sugerencias, proponer_sql	Sugerencia
ValidadorController	Coordina el flujo de validación.	request, usuario	procesar, guardar, responder	Vista, Core
ApiResponse	Estandariza respuesta JSON.	success, data, error	success, error	API
Historial	Guarda resultados por usuario.	id, usuarioId, fecha	registrar, listar, filtrar	Usuario
Reporte	Genera evidencia descargable.	tipo, contenido	generar_pdf, generar_sql	Validacion

H. Diagramas de Actividades, Secuencia y Clases

El flujo principal inicia con el ingreso del esquema, continúa con validación de formato, parsing, diagnóstico, generación de sugerencias, descarga de reportes y persistencia del historial.



Figura 8. Diagrama de actividad del proceso de validación.

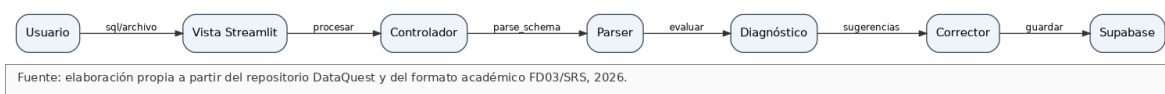


Figura 9. Diagrama de secuencia principal.

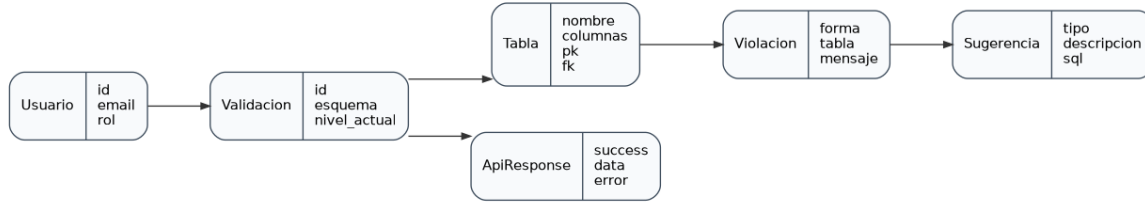


Figura 10. Diagrama de objetos/clases conceptuales.

V. API de integración y endpoints

La API de integración se implementa con FastAPI y expone servicios JSON para validar esquemas relacionales. Su endpoint principal recibe el campo sql con sentencias CREATE TABLE, ejecuta el parser y devuelve el diagnóstico estructurado.

Método	Endpoint	Entrada	Salida	Estado
GET	/	Sin cuerpo	status, message, docs_url	Implementado
POST	/api/normalizacion	{ sql: string }	success, nivel_actual, violaciones_1fn, violaciones_2fn, violaciones_3fn, mejoras_opcionales, resumen	Implementado
POST	/auth/login	email, password	token, usuario, rol	Planificado/Parcial
GET	/historial	token usuario	lista de validaciones	Planificado/Parcial
GET	/reportes/{id}	id validación	PDF/SQL generado	Planificado
POST	/api/normalizacion/batch	lista de esquemas	diagnósticos múltiples	Planificado

VI. Matriz de trazabilidad

Req.	Módulo	Caso de uso	Entidad / Objeto	Endpoint / Evidencia	Criterio de aceptación
RF-02	Web	CU02	Validacion	views/04_validador.py	Permite ingresar SQL y enviarlo al motor.
RF-03	Core	CU03	Parser	core/parser.py	Extrae tablas válidas del SQL.
RF-04	Core	CU03	Diagnostico	core/validador_1fn.py	Reporta incumplimientos de 1FN.
RF-05	Core	CU03	Diagnostico	core/validador_2fn.py	Detecta dependencias parciales.
RF-06	Core	CU03	Diagnostico	core/validador_3fn.py	Detecta dependencias transitivas.
RF-08	Core	CU04	Corrector	core/corrector.py	Genera sugerencias entendibles.
RF-10	API	CU07	ApiResponse	POST /api/normalizacion	Devuelve JSON válido.



RF-11	API	CU07	ApiResponse	api.py	Incluye success, nivel y violaciones.
RF-15	IDE/API	CU07	Cliente VS Code	vscode-extension	Inserta reporte en el editor.
RF-17	DB/Web	CU06	Historial	Supabase/PostgreSQL	Lista validaciones del usuario.

VII. Criterios de calidad, seguridad y aceptación

A. Criterios de calidad

- El diagnóstico debe ser reproducible con el mismo SQL de entrada.
- La respuesta API debe tener estructura JSON estable y documentada.
- El sistema no debe mezclar lógica de interfaz con lógica de normalización.
- Las tablas y reportes del documento deben guardar trazabilidad con módulos reales del repositorio.
- Los errores deben presentarse de manera comprensible para estudiantes y evaluadores.

B. Criterios de seguridad

- No exponer credenciales de Supabase ni variables de entorno en la interfaz, reportes o repositorio.
- Evitar mostrar stacktrace al usuario final en respuestas de API.
- Aplicar control de acceso para historial y datos privados.
- Validar contenido de archivos antes de procesarlos.
- Mantener CORS y autenticación bajo revisión antes de un despliegue productivo.

C. Criterios de aceptación

ID	Criterio	Condición de aceptación
CA-01	Validación SQL	Al enviar CREATE TABLE válido, el sistema devuelve nivel actual y listas de violaciones.
CA-02	Entrada vacía	Al enviar sql vacío, la API responde error controlado con code 400.
CA-03	Sin tablas	Al enviar texto sin CREATE TABLE, la API informa que no encontró tablas válidas.
CA-04	3FN	Si existe dependencia transitiva, se registra en violaciones_3fn.
CA-05	Web	La interfaz muestra resultado, resumen y sugerencias sin romper el diseño.
CA-06	VS Code	La extensión puede enviar SQL y anexar reporte al documento.
CA-07	Historial	El usuario autenticado puede consultar validaciones propias.
CA-08	Documentación	El FD03 mantiene formato académico, diagramas, tablas y trazabilidad.



CONCLUSIONES

- DataQuest se define como una solución académica coherente para validar normalización de bases de datos relacionales mediante una arquitectura modular.
- La separación entre interfaz, controladores, motor de análisis, API y persistencia mejora mantenibilidad, pruebas e integración.
- El endpoint de normalización permite validar el motor con Postman, clientes externos y extensión VS Code.
- La documentación FD03 permite vincular requerimientos, módulos, casos de uso, objetos, endpoints y criterios de aceptación.
- El sistema debe continuar diferenciando funciones implementadas, parciales y planificadas para evitar contradicciones en revisión docente.

RECOMENDACIONES

- Fortalecer pruebas unitarias para parser, validadores 1FN/2FN/3FN y corrector.
- Evitar exposición de stacktrace en respuestas productivas y registrar errores solo en logs internos.
- Completar autenticación, historial y políticas de acceso RLS antes de publicar datos reales de usuarios.
- Mantener la API documentada con ejemplos de request, response y errores.
- Actualizar el FD03 cada vez que cambie el alcance técnico o se incorporen módulos nuevos.

BIBLIOGRAFÍA

Elmasri, R. y Navathe, S. B. Fundamentos de Sistemas de Bases de Datos.
Date, C. J. Introducción a los Sistemas de Bases de Datos.
Silberschatz, A., Korth, H. y Sudarshan, S. Database System Concepts.
IEEE. Recommended Practice for Software Requirements Specifications.